

Machine Learning in Practice I

Lecture 9

Antal Jakovac, 2025

Linear regression fitting in a functional basis reliability, overfitting inter- and extrapolation



fitting in a functional basis

- in lot of applications the task is to fit a function in a basis
- one-dimensional case: $f(t)$ function, $g_\alpha(t)$ basis, and we seek a_α :

$$f(t) = \sum_{\alpha=1}^{N_{basis}} a_\alpha g_\alpha(t)$$

- examples:
 - polynomial expansion: $g_\alpha(t) = t^\alpha \rightarrow$ linear fit uses 2D basis $g = (1, t)$
 - Fourier expansion: $g_\alpha(t) = e^{i\alpha t}$
 - Laplace transformation: $g_\alpha(t) = e^{-\alpha t}$

fitting in a functional basis

- the basis elements should be linearly independent, i.e.

$$0 = \sum_{\alpha=1}^{N_{basis}} a_{\alpha} g_{\alpha}(t) \Leftrightarrow a_{\alpha} = 0$$

- in practice we have data pairs $D = \{(t_n, y_n) \mid n=1, 2, \dots, N\}$

- solve $y_n \approx \sum_{\alpha=1}^{N_{basis}} a_{\alpha} g_{\alpha}(t_n)$, which means minimize $\chi^2 = \sum_{n=1}^{N_{data}} \frac{(y_n - \sum_{\alpha} a_{\alpha} g_{\alpha}(t_n))^2}{\sigma_n^2}$

- once we get the coefficients, we can interpolate: $f(t) \approx \sum_{\alpha=1}^{N_{basis}} a_{\alpha} g_{\alpha}(t)$
(prediction)

fitting in a functional basis

- computation (*more elegant than the linear case ...*)
 - ➔ introduce vector notation

$$X \in \mathbb{R}^{N \times N_{\text{basis}}}, \quad X_{n\alpha} = g_{\alpha}(t_n), \quad Y \in \mathbb{R}^N, \quad c \in \mathbb{R}^{N_{\text{basis}}} \Rightarrow \sum_{\alpha} a_{\alpha} g_{\alpha}(t_n) = X \cdot a$$

$$C \in \mathbb{R}^{N \times N}, \quad C = \text{diagonal}(\sigma_1, \sigma_2, \dots, \sigma_N)$$

$$\chi^2 = \sum_{n=1}^{N_{\text{data}}} \frac{\left(Y_n - \sum_{\alpha} a_{\alpha} X_{n\alpha}\right)^2}{\sigma_n^2} = (X \cdot a - Y)^T C^{-1} (X \cdot a - Y)$$

fitting in a functional basis

- computation (*more elegant than the linear case ...*)
 - extremum: $\frac{\partial X^2}{\partial a} = 0 = X^T C^{-1} (X \cdot a - Y)$
 - solution: $a = (X^T C^{-1} X)^{-1} X^T C^{-1} Y$ ($\rightarrow$ c.f. linear case)
 - for uniform correlation matrix: $a = (X^T X)^{-1} X^T Y$

fitting in a functional basis

- example:

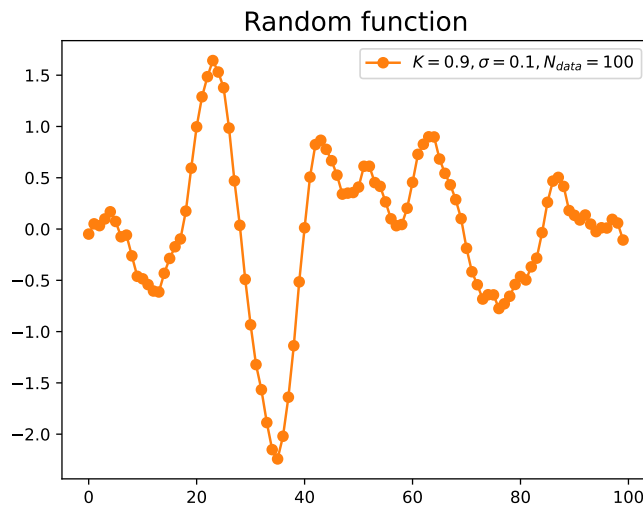
- ➔ create a random function, e.g. damped oscillator with noise

$$x_{n+1} = K(2x_n - x_{n-1} + \sigma \xi_n)$$

- ➔ choose functional basis, e.g. sin-cos

$$g_\alpha(t) = (\sin(\alpha t), \cos(\alpha t))$$

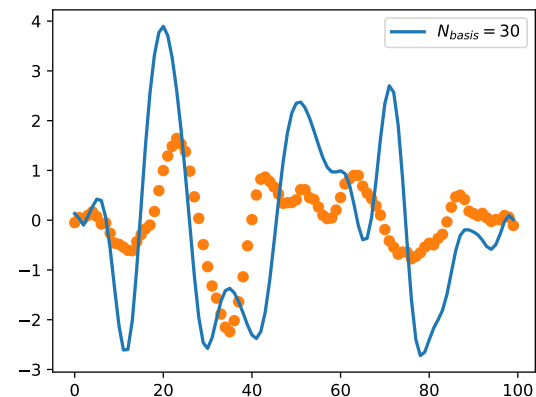
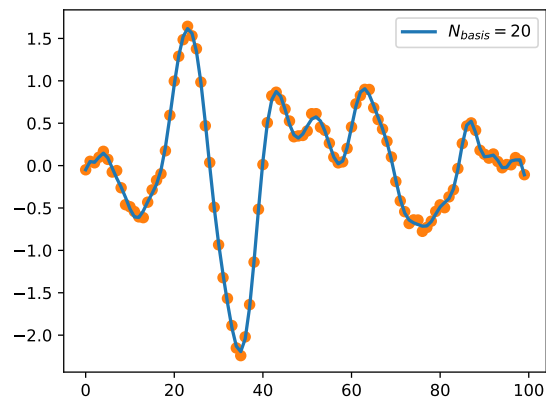
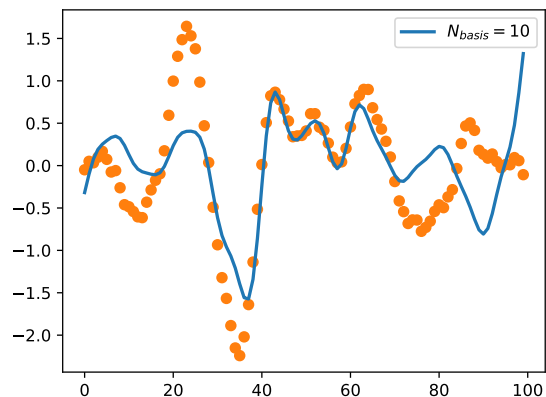
- ➔ use the fit formulae



fitting in a functional basis

- example:

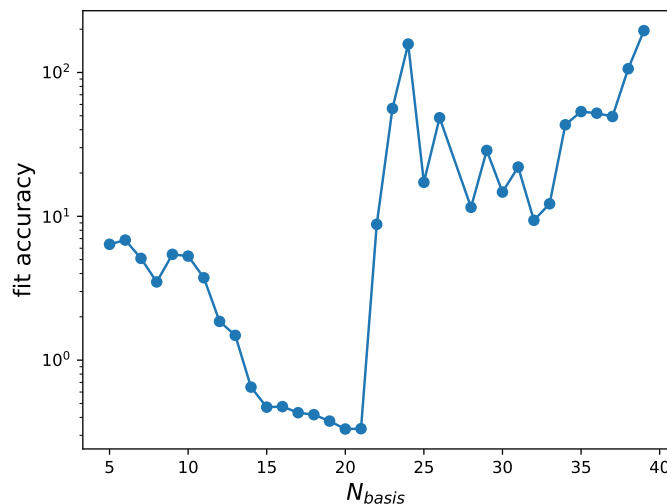
→ result depends on the number of basis elements → non-monotonic!



fitting in a functional basis

- example:

- ➡ result depends on the number of basis elements → non-monotonic!





fitting in a functional basis

- what happened?
 - we have to invert a matrix: $a = (X^T X)^{-1} X^T Y$
 - what if there are small eigenvalues?

eigenvalues and eigenvectors in nutshell

- eigenvectors are stabile directions of a matrix

$$M v_n = \lambda_n v_n \quad V_{in} = (v_n)_i \Rightarrow MV = V \Lambda, \quad \Lambda = \text{diagonal}(\lambda_1, \lambda_2, \dots, \lambda_{N_{\text{basis}}})$$

- eigenvectors form a complete basis (*in most numerical cases*)

$$M = V \Lambda V^{-1} \Rightarrow M^{-1} = V \Lambda^{-1} V^{-1}$$

- matrix functions, defined with Laurent series

$$f(M) = \sum_k c_k M^k = \sum_k c_k (V \Lambda V^{-1})^k = V \left(\sum_k c_k \Lambda^k \right) V^{-1} = V f(\Lambda) V^{-1}$$

eigenvalues and eigenvectors in nutshell – special cases

- hermitian matrices: $M = M^+ = M^{*T} \rightarrow$ real symmetric matrices
 - eigenvalues are real
 - eigenvectors are orthogonal $V^+ V = V^+ V = 1, \quad M = V \Lambda V^+$
- $M = X^+ X$ matrices:
 - hermitian
 - eigenvalues are non-negative (positive definite)

$$\begin{aligned} M v_n = \lambda_n v_n &\Rightarrow v_n^+ M = \lambda_n^* v_n^+ \Rightarrow v_n^+ M v_m = \lambda_n^* v_n^+ M v_m = \lambda_m v_n^+ M v_m \\ X^+ X v_n = \lambda_n v_n &\Rightarrow \lambda_n = v_n^+ X^+ X v_n = |X v_n|^2 \end{aligned}$$



fitting in a functional basis

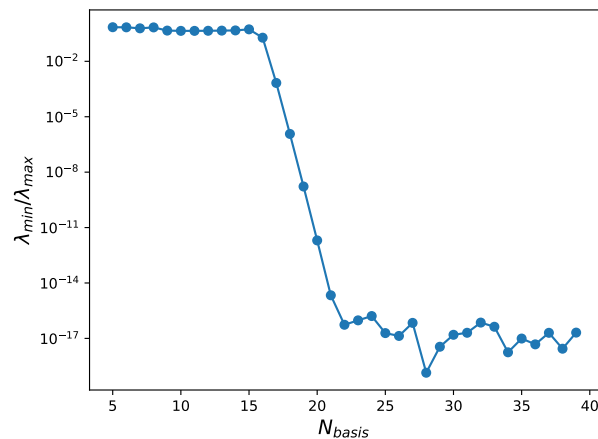
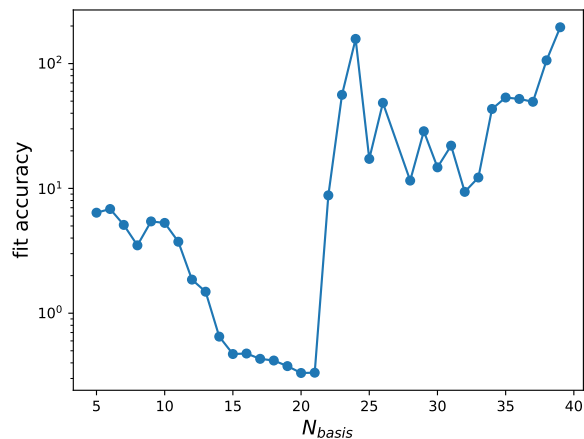
- what happened?
 - we have to invert a matrix: $a = (X^T X)^{-1} X^T Y$
 - what if there are small eigenvalues?
- a linear equation $Mx = y \Rightarrow x = M^{-1} y \rightarrow$ how reliable is it?
- add an eigenvector to the solution! $M(x + z v_0) = y + \lambda_0 z v_0$
- if numerical tolerance is $\epsilon \rightarrow |y + \lambda_0 z v_0| \approx |y| + \lambda_0 z < (1 + \epsilon)|y| \Rightarrow z < \frac{\epsilon}{\lambda} |y|$
- if $\lambda \ll \epsilon \Rightarrow z$ can be very large!

fitting in a functional basis

- typical value: $\epsilon \approx 10^{-12} - 10^{-17}$
 - measurement errors
 - numerical calculation errors
- ***ill-conditioned matrices***: there are eigenvalues $\lambda \ll 10^{-15} \lambda_{max}$
the inverse is not reliable!
- very common in higher dimensions! → in ML or AI
- never use the naive linear equation solving formulae!

fitting in a functional basis

- in our example: for $N_{basis} \approx 21$ we reached Python's precision $\epsilon \sim 10^{-17}$
- after that value the fitting precision is drastically reduced





fitting in a functional basis

- what can we do with an ill-conditioned matrix?
 - pseudo-inverse
 - regularization

pseudo-inverse

- assume we can add the smallest eigenvector(s) with arbitrary coefficients
- choose that value that leads to the smallest length

$$x = \sum_n \alpha_n v_n \Rightarrow |x + c v_1|^2 = (\alpha_1 + c)^2 + \sum_{n=2}^{N_{basis}} \alpha_n^2 = \text{minimal} \Rightarrow c = -\alpha_1$$

- lesson: we should leave out all small eigenvectors!

- in formula:
$$M_{ij}^{-1} = \sum_{n=1}^{N_{basis}} \frac{1}{\lambda_n} v_{ni} v_{nj} \Rightarrow \sum_{\lambda_n > \epsilon \lambda_{max}}^{N_{basis}} \frac{1}{\lambda_n} v_{ni} v_{nj}$$

pseudo-inverse

- in Python code

```
# pseudoinverse fitting
```

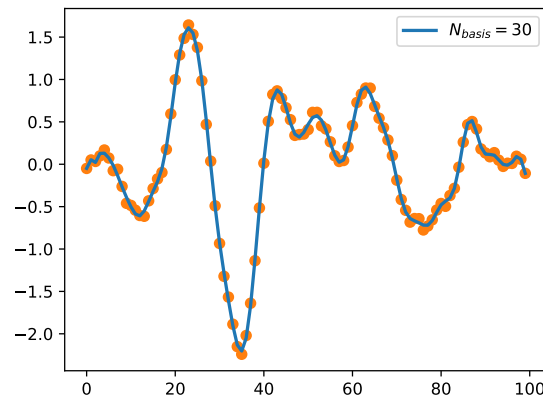
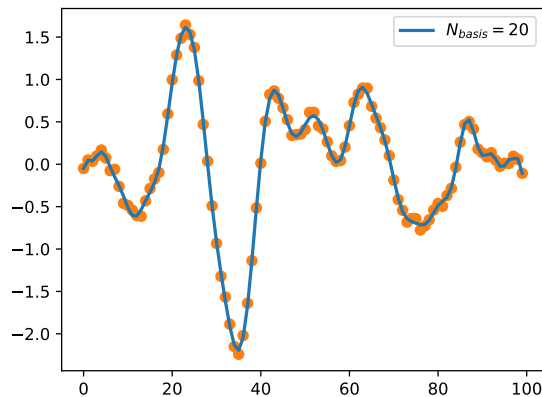
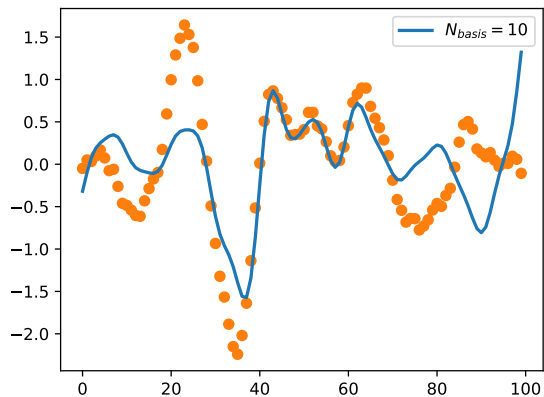
```
nus = np.linspace(1,30,Nnu)
```

```
base1 = [[np.cos(np.pi*nu*n/Ndata) for n in range(Ndata)] for nu in nus]
```

```
base2 = [[np.sin(np.pi*nu*n/Ndata) for n in range(Ndata)] for nu in nus]
```

```
f = np.array(base1+base2).T
```

```
c = np.linalg.pinv(f.T@f)@f.T@y  
plt.plot(f@c)
```





- pseudo-inverse works well, but closely connected to linear algebra
- more general method: **regularization**
- idea: add extra term to the loss function
 - stabilize result
 - spoils close connection to data
 - tradeoff: should use in a balanced manner
- most frequently used: L1, L2 regularization, MEM

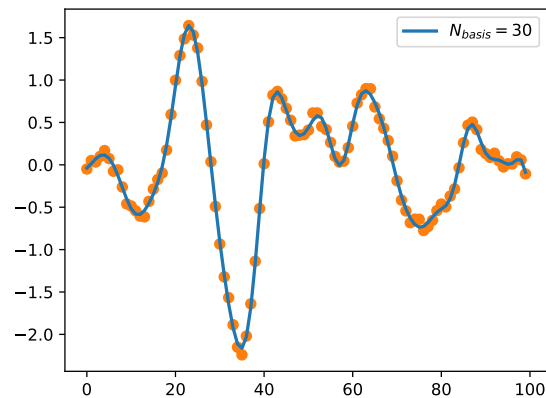
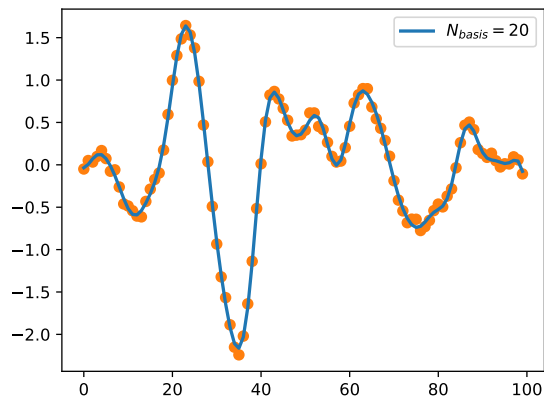
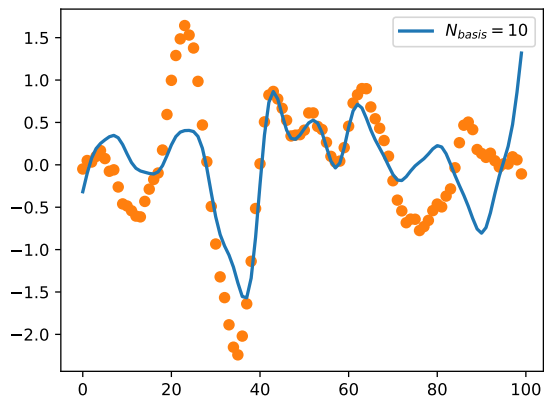
- L2 regularized loss

$$L = \sum_{n=1}^{N_{data}} \frac{(y_n - \sum_{\alpha} a_{\alpha} x_{n\alpha})^2}{\sigma_n^2} + \lambda \sum_{\alpha=1}^{N_{basis}} a_{\alpha}^2 \Rightarrow (X \cdot a - Y)^T C^{-1} (X \cdot a - Y) + \lambda a^T a$$

$$\frac{\partial L}{\partial a} = 0 = X^T C^{-1} (X \cdot a - Y) + \lambda a \Rightarrow a = (X^T C^{-1} X + \lambda)^{-1} X^T C^{-1} Y$$

- formally: add a small number to the matrix to be inverted
- works only for positive definite matrices (true in our case)

- L2 regularized loss results

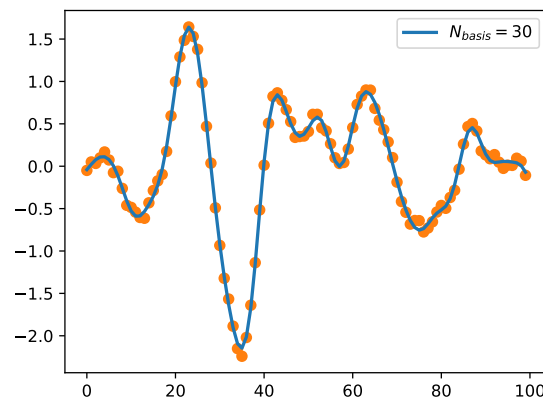
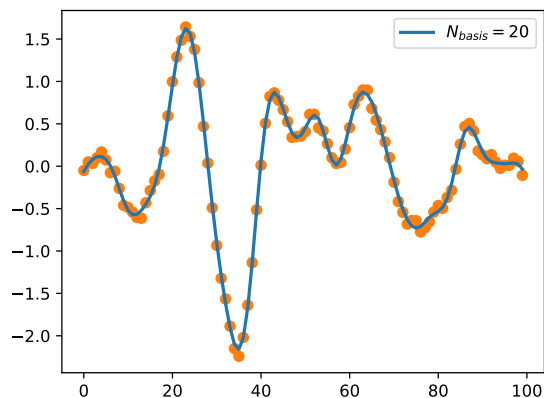
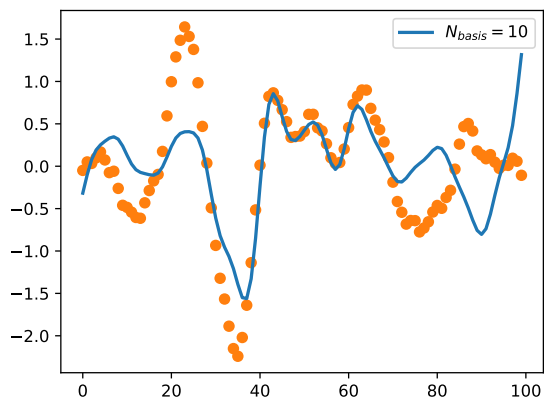


Linear regression

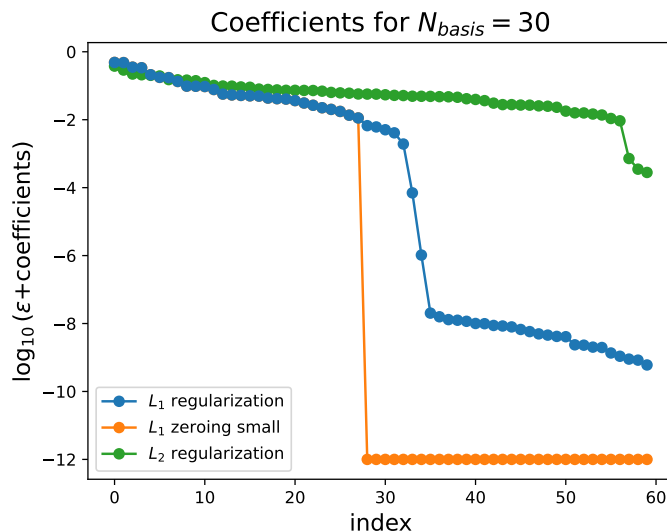
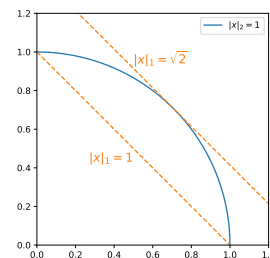
- L1 regularized loss – also for positive definite matrices

$$L = \sum_{n=1}^{N_{data}} \frac{\left(y_n - \sum_{\alpha} a_{\alpha} x_{n\alpha}\right)^2}{\sigma_n^2} + \lambda \sum_{\alpha=1}^{N_{basis}} |a_{\alpha}|$$

- minimization with numerical methods



- L1 regularization benefit: likes small coefficients
- reason: $x^2 + y^2 = 1 \Rightarrow \min(|x| + |y|) \Rightarrow x=0, y=1$ or equivalent
- can be used to eliminate features $\rightarrow$ c.f. previous plot



END OF LECTURE 9